

PANDAS

The Complete Developer Guide

Data Analysis & Manipulation with Python

pandas • DataFrames • Series • Real-World Use Cases

v2.x | Python 3.10+ | 2026 Edition

1. What is Pandas?

Pandas is an open-source, BSD-licensed Python library that provides high-performance, easy-to-use data structures and data analysis tools. Built on top of NumPy, it is the de facto standard for data manipulation and exploratory data analysis (EDA) in Python.

■ The name 'Pandas' is derived from 'Panel Data' — a term from econometrics for multi-dimensional structured data sets.

1.1 Core Design Goals

Goal	Description
Flexibility	Handle structured/semi-structured data with ease
Performance	Vectorised operations via NumPy C extensions
Expressiveness	Intuitive, SQL-like querying and transformation
Integration	Works seamlessly with NumPy, Matplotlib, Scikit-learn, PyTorch
I/O Rich	Read/write CSV, Excel, JSON, SQL, Parquet, HDF5 and more

1.2 Installation

```
# Using pip
pip install pandas

# Using conda
conda install pandas

# Verify installation
import pandas as pd
print(pd.__version__) # e.g. 2.2.1
```

1.3 Why Pandas over Plain Python?

- Built-in handling of missing data (NaN / NaT).
- Automatic alignment of data by label during operations.
- Intelligent label-based slicing, fancy indexing, and subsetting.
- Aggregation and transformation of large datasets in a few lines.
- Time-series specific functionality (resampling, rolling windows).
- Efficient merge / join operations comparable to SQL.

2. DataFrame vs Series

2.1 Series — 1-D Labelled Array

A Series is a one-dimensional labelled array capable of holding any data type (integers, strings, floats, Python objects, etc.). The axis labels are collectively referred to as the index.

```
import pandas as pd

# Create from a list
s = pd.Series([10, 20, 30, 40], name='Sales')
print(s)
# 0    10
# 1    20
# 2    30
# 3    40
# Name: Sales, dtype: int64

# Create with custom index
s2 = pd.Series({'Q1': 150, 'Q2': 200, 'Q3': 180, 'Q4': 220})
print(s2['Q2'])  # 200

# Arithmetic is vectorised
print(s2 * 1.1)  # Apply 10% increase to all quarters
```

Key Series Attributes

Attribute / Method	Description	Example
s.index	Index labels	RangeIndex(0, 4)
s.values	NumPy array of values	array([10,20,30,40])
s.dtype	Data type	int64
s.shape	Tuple (n,)	(4,)
s.name	Series name	'Sales'
s.describe()	Summary statistics	count, mean, std...

2.2 DataFrame — 2-D Labelled Table

A DataFrame is a two-dimensional, size-mutable, heterogeneous tabular data structure with labelled axes (rows and columns). You can think of it as a dictionary of Series objects sharing the same index, or as a Python equivalent of a SQL table / Excel spreadsheet.

```
import pandas as pd

data = {
    'Name':    ['Alice', 'Bob', 'Charlie', 'Diana'],
    'Age':     [28, 34, 22, 31],
    'City':    ['Chennai', 'Mumbai', 'Delhi', 'Bangalore'],
    'Salary':  [85000, 92000, 67000, 110000]
}

df = pd.DataFrame(data)
print(df)

#      Name  Age   City  Salary
# 0   Alice   28  Chennai   85000
# 1    Bob   34   Mumbai   92000
# 2  Charlie   22    Delhi   67000
# 3   Diana   31  Bangalore  110000

# Access a column → Series
print(df['Salary'])           # Series

# Access a row by label
print(df.loc[1])              # Bob's row

# Access by position
print(df.iloc[0, 2])          # 'Chennai'
```

2.3 Visual Comparison

Feature	Series	DataFrame
Dimensions	1-D	2-D
Analogy	A single column in Excel	Full Excel worksheet
Index	One index (row labels)	Row index + column labels
Data types	Homogeneous (one dtype)	Heterogeneous (per column)
Created from	list, dict, ndarray	dict of Series / lists, CSV, SQL
Slicing result	Scalar or Series	Series or DataFrame
Math ops	Element-wise on values	Element-wise, column-aligned

3. Pandas Core Concepts

3.1 Index & Alignment

Every pandas object carries an Index. When you perform arithmetic or merge operations, pandas automatically aligns data by index label — eliminating entire classes of bugs that arise from positional misalignment.

```
s1 = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
s2 = pd.Series([10, 20, 30], index=['b', 'c', 'd'])
print(s1 + s2)
# a      NaN    ← no match → NaN
# b     12.0
# c     23.0
# d      NaN    ← no match → NaN
```

3.2 Missing Data (NaN / NaT)

```
import numpy as np
df = pd.DataFrame({'A': [1, np.nan, 3], 'B': [np.nan, 5, 6]})
df.isnull()          # Boolean mask of missing values
df.dropna()          # Drop rows with any NaN
df.fillna(0)         # Replace NaN with 0
df.fillna(df.mean()) # Replace NaN with column mean
df.interpolate()     # Linear interpolation
```

3.3 Vectorised Operations & Broadcasting

Pandas operations are vectorised (executed in compiled C/Cython code), making them orders of magnitude faster than Python loops.

```
df['Salary_INR'] = df['Salary'] * 83      # scalar broadcast
df['Tax']        = df['Salary'] * 0.30    # column-wise
df['Net']        = df['Salary'] - df['Tax']

# apply() for custom functions
df['Grade'] = df['Salary'].apply(lambda x: 'A' if x > 90000 else 'B')
```

3.4 GroupBy — Split → Apply → Combine

The GroupBy mechanism implements the split-apply-combine strategy popularised by Hadley Wickham. It allows aggregating, transforming, and filtering groups of data independently.

```
# Group by City, compute mean salary
df.groupby('City')['Salary'].mean()

# Multiple aggregations
df.groupby('City').agg({'Salary': ['mean', 'max', 'count']})

# Transform (returns same shape as input)
df['Salary_vs_CityAvg'] = df.groupby('City')['Salary'].transform(
    lambda x: x - x.mean()
)

# Filter groups
df.groupby('City').filter(lambda g: g['Salary'].mean() > 80000)
```

3.5 Merge, Join & Concat

```
# pd.merge – SQL-style joins
orders = pd.DataFrame({'order_id':[1,2,3], 'customer_id':[10,11,10], 'amount':[500,300,700]})
customers = pd.DataFrame({'customer_id':[10,11], 'name':['Alice','Bob']})

# Inner join
pd.merge(orders, customers, on='customer_id')

# Left join
pd.merge(orders, customers, on='customer_id', how='left')

# pd.concat – stacking DataFrames
combined = pd.concat([df1, df2], ignore_index=True)           # row-wise
wide = pd.concat([df1, df2], axis=1)                         # column-wise
```

3.6 Pivot Tables & Crosstab

```
# Pivot table
pivot = df.pivot_table(
    values='Salary',
    index='City',
    columns='Grade',
    aggfunc='mean',
    fill_value=0
)

# Crosstab – frequency table
pd.crosstab(df['City'], df['Grade'])
```

4. Common Functions with Examples

4.1 Reading & Writing Data

```
# Read CSV
df = pd.read_csv('customers_1000.csv')

# Read Excel
df = pd.read_excel('report.xlsx', sheet_name='Sheet1')

# Read from SQL
import sqlalchemy
engine = sqlalchemy.create_engine('oracle+cx_oracle://user:pass@host/db')
df = pd.read_sql('SELECT * FROM orders WHERE status = :s',
                 engine, params={'s': 'OPEN'})

# Write outputs
df.to_csv('output.csv', index=False)
df.to_excel('output.xlsx', index=False)
df.to_parquet('output.parquet') # columnar, fast for large data
```

4.2 Exploration Functions

```
df.head(5)      # First 5 rows
df.tail(5)      # Last 5 rows
df.shape        # (rows, cols)
df.info()       # Dtypes + memory usage
df.describe()   # Stats: count, mean, std, min, max, percentiles
df.dtypes       # Data type of each column
df.columns      # Column names
df.nunique()    # Unique value count per column
df.value_counts('Country') # Frequency of each value
```

4.3 Selection & Filtering

```
# Column selection
df['Name']                # Single column → Series
df[['Name', 'Email', 'Country']]  # Multiple columns → DataFrame

# Row filtering
df[df['Country'] == 'India']
df[(df['Country'] == 'India') & (df['City'] == 'Chennai')]

# .loc → label-based
df.loc[0:4, 'Name':'City']

# .iloc → integer position
df.iloc[0:5, 0:3]

# Query method (SQL-like)
df.query("Country == 'India' and City == 'Mumbai'")

# Boolean with isin()
df[df['Country'].isin(['India', 'Japan', 'Germany'])]
```

4.4 String Operations (str accessor)

```
df['Email'].str.contains('@gmail')
df['First Name'].str.upper()
df['Email'].str.split('@').str[1]  # Extract domain
df['Full Name'] = df['First Name'].str.cat(df['Last Name'], sep=' ')
df['Phone 1'].str.replace('-', '').str.strip()
df['Country'].str.startswith('U')  # United States, United Kingdom...
```

4.5 Sorting & Ranking

```
df.sort_values('Salary', ascending=False)  # Descending salary
df.sort_values(['Country', 'City'])        # Multi-column sort
df['Rank'] = df['Salary'].rank(ascending=False, method='dense')
df.nlargest(10, 'Salary')                  # Top 10 earners
df.nsmallest(5, 'Age')                    # 5 youngest
```

4.6 Date & Time


```
df['Date'] = pd.to_datetime(df['Date'])
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month_name()
df['DOW'] = df['Date'].dt.day_name()

# Resample time-series data
df.set_index('Date').resample('M')['Sales'].sum()

# Rolling window
df['7d_avg'] = df['Sales'].rolling(window=7).mean()
```

4.7 Window Functions

```
# Cumulative operations
df['Cumulative_Sales'] = df['Sales'].cumsum()
df['Running_Max'] = df['Sales'].cummax()

# Rolling statistics
df['Rolling_Mean_7d'] = df['Sales'].rolling(7).mean()
df['Rolling_Std_30d'] = df['Sales'].rolling(30).std()

# Expanding window
df['Expanding_Sum'] = df['Sales'].expanding().sum()
```

4.8 Aggregation & Transform

```
# agg with named results
df.groupby('Country').agg(
    total_customers=('Customer Id', 'count'),
    avg_salary=('Salary', 'mean'),
    max_salary=('Salary', 'max')
)

# Multiple functions at once
df['Salary'].agg(['mean', 'median', 'std', 'min', 'max'])

# Normalize within groups using transform
df['NormSalary'] = df.groupby('Country')['Salary'].transform(
    lambda x: (x - x.mean()) / x.std()
)
```

5. Real-World Use Cases

5.1 Clinical Trial Supply Analytics (Pharma / eCTS)

Pandas is widely used in regulated pharma environments to analyse supply chain data, monitor inventory levels across sites, and detect anomalies before they become incidents.

```
import pandas as pd

# Load site-level inventory snapshot
df = pd.read_csv('ctms_inventory.csv', parse_dates=['snapshot_date'])

# Filter active studies with low stock
low_stock = df[
    (df['status'] == 'ACTIVE') &
    (df['quantity_on_hand'] < df['min_threshold'])
]

# Summarise by study and country
summary = low_stock.groupby(['study_id', 'country']).agg(
    sites_at_risk=('site_id', 'nunique'),
    total_shortage=('shortage_units', 'sum')
).reset_index()

# Identify studies crossing critical threshold
critical = summary[summary['sites_at_risk'] >= 3]
critical.to_excel('critical_sites_report.xlsx', index=False)
```

5.2 ServiceNow RITM Processing Analytics

```
import pandas as pd

df = pd.read_json('servicenow_ritm_export.json')
df['opened_at'] = pd.to_datetime(df['opened_at'])
df['closed_at'] = pd.to_datetime(df['closed_at'])

# Calculate resolution time in hours
df['resolution_hrs'] = (df['closed_at'] - df['opened_at']).dt.total_seconds() / 3600

# SLA breach detection (24-hr SLA)
df['sla_breached'] = df['resolution_hrs'] > 24

# Weekly breach rate
weekly = df.set_index('opened_at').resample('W').agg(
    total=('number', 'count'),
    breached=('sla_breached', 'sum')
)

weekly['breach_rate_%'] = (weekly['breached'] / weekly['total'] * 100).round(2)
```

5.3 E-Commerce Customer Analytics

```
# RFM Analysis – Recency, Frequency, Monetary
import pandas as pd
from datetime import date

df = pd.read_csv('orders.csv', parse_dates=['order_date'])
today = pd.Timestamp(date.today())

rfm = df.groupby('customer_id').agg(
    recency=('order_date', lambda x: (today - x.max()).days),
    frequency=('order_id', 'count'),
    monetary=('amount', 'sum')
).reset_index()

# Score each dimension 1-5 using quantile bins
rfm['R'] = pd.qcut(rfm['recency'], q=5, labels=[5,4,3,2,1]).astype(int)
rfm['F'] = pd.qcut(rfm['frequency'], q=5, labels=[1,2,3,4,5]).astype(int)
rfm['M'] = pd.qcut(rfm['monetary'], q=5, labels=[1,2,3,4,5]).astype(int)
rfm['RFM_Score'] = rfm['R'] + rfm['F'] + rfm['M']

# Segment customers
rfm['Segment'] = pd.cut(rfm['RFM_Score'],
    bins=[0, 6, 9, 12, 15],
    labels=['At Risk', 'Needs Attention', 'Loyal', 'Champions'])
```

5.4 Log Aggregation & Anomaly Detection (Observability)

```
# Parse structured JSON logs exported from Grafana Loki
import pandas as pd

df = pd.read_json('loki_export.ndjson', lines=True)
df['timestamp'] = pd.to_datetime(df['timestamp'])
df.set_index('timestamp', inplace=True)

# Error rate per minute
error_rate = df[df['level'] == 'error'].resample('1min')['event'].count()

# Rolling Z-score anomaly detection
rolling_mean = error_rate.rolling(10).mean()
rolling_std = error_rate.rolling(10).std()
z_scores = (error_rate - rolling_mean) / rolling_std
anomalies = error_rate[z_scores.abs() > 2.5]
print(f'Anomalous minutes detected: {len(anomalies)}')
```

6. Pandas vs NumPy vs PyTorch

These three libraries form the Python data science stack. Each occupies a distinct layer of the pipeline. Understanding their roles prevents misuse and performance pitfalls.

Feature	Pandas	NumPy	PyTorch
Primary purpose	Data manipulation & EDA	Numerical computing arrays	Deep learning & autograd
Core data structure	DataFrame / Series	ndarray	Tensor
Labelled data	Yes (index + columns)	No (positional only)	No
Heterogeneous dtypes	Yes (per column)	No (one dtype per array)	No (one dtype per tensor)
Missing data	First-class (NaN/NaT/NA)	Limited (np.nan only)	Not natively supported
String support	Full .str accessor	Object arrays (slow)	None
GPU support	No (CPU only)	No (CPU only)	Yes (CUDA/MPS)
Automatic gradient	No	No	Yes (autograd engine)
I/O formats	CSV, Excel, SQL, JSON, Parquet...	npz, npy, text files	pt/pth checkpoints
Time series	Rich (resample, rolling...)	Manual indexing	Not built-in
Typical usage	Data cleaning, aggregation, EDA, feature engineering	Linear algebra, array math, statistics, image arrays	Model training, inference, neural networks
Interop	→ NumPy via .values/.to_numpy() → Tensor via torch.from_numpy() → NumPy via .numpy()		
Performance (CPU)	Good (vectorised)	Excellent (C extensions)	Good (optimised kernels)
Learning curve	Low–Medium	Low	Medium–High
Primary users	Data analysts, Data engineers	Scientists, ML researchers	ML engineers, Researchers

6.1 The Typical Pipeline

Raw Data (CSV/SQL/API) → Pandas (clean, transform, feature-engineer) → NumPy (matrix ops, normalise) → PyTorch (model training, inference) → Pandas (analyse results, export report)

```
import pandas as pd, numpy as np, torch

# 1. Load & clean with Pandas
df = pd.read_csv('features.csv').dropna()
df['age_norm'] = (df['age'] - df['age'].mean()) / df['age'].std()

# 2. Convert to NumPy for matrix operations
X = df[['age_norm', 'salary', 'tenure']].to_numpy(dtype='float32')
y = df['churn'].to_numpy(dtype='float32')

# 3. Convert to PyTorch Tensors for model training
X_tensor = torch.from_numpy(X)
y_tensor = torch.from_numpy(y)

# 4. After inference, back to Pandas for analysis
preds = model(X_tensor).detach().numpy()
df['predicted_churn'] = preds
df.groupby('segment')['predicted_churn'].mean().plot(kind='bar')
```

6.2 When to Choose Which?

Scenario	Best Tool
Explore a CSV/Excel dataset	Pandas
Clean messy data, handle NaNs	Pandas
Join multiple tables (SQL-like)	Pandas
Time-series resampling / rolling	Pandas
Fast matrix multiplication	NumPy
Statistical simulations	NumPy
Pass features to sklearn	NumPy (.to_numpy())
Train a neural network	PyTorch
Run on GPU	PyTorch (CUDA)
Fine-tune a transformer (LLM)	PyTorch + HuggingFace
Feature engineering before ML	Pandas → NumPy → PyTorch

7. Quick Reference Cheat Sheet

I/O

Function / Method	Description
<code>read_csv / to_csv</code>	Read/write CSV
<code>read_excel / to_excel</code>	Read/write Excel
<code>read_sql</code>	Read from SQL DB
<code>read_parquet</code>	Read columnar Parquet file
<code>read_json</code>	Read JSON / NDJSON

Inspection

Function / Method	Description
<code>df.head(n)</code>	First n rows
<code>df.info()</code>	Dtype + memory
<code>df.describe()</code>	Summary statistics
<code>df.shape</code>	(rows, cols)
<code>df.nunique()</code>	Unique count per column

Selection

Function / Method	Description
<code>df['col']</code>	Select column
<code>df[['a', 'b']]</code>	Select multiple columns
<code>df.loc[r, c]</code>	Label-based selection
<code>df.iloc[r, c]</code>	Position-based selection
<code>df.query(expr)</code>	SQL-like filter

Cleaning

Function / Method	Description
<code>df.dropna()</code>	Remove NaN rows
<code>df.fillna(v)</code>	Fill NaN with value
<code>df.drop_duplicates()</code>	Remove duplicate rows
<code>df.rename(cols)</code>	Rename columns
<code>df.astype(dtype)</code>	Cast column types

Transform

Function / Method	Description
<code>df.apply(fn)</code>	Apply function row/col
<code>df.map(fn)</code>	Element-wise map
<code>df.assign(**kw)</code>	Add new columns
<code>pd.get_dummies()</code>	One-hot encode
<code>df.clip(lo, hi)</code>	Clip values to range

Aggregation

Function / Method	Description
<code>df.groupby(col)</code>	Group by column(s)
<code>df.agg({...})</code>	Named aggregations
<code>df.pivot_table()</code>	Pivot with aggregation
<code>pd.crosstab()</code>	Frequency table
<code>df.resample(rule)</code>	Time-series resampling

Merge / Join

Function / Method	Description
<code>pd.merge(l, r)</code>	SQL-style join
<code>pd.concat([...])</code>	Stack DataFrames
<code>df.join(other)</code>	Join on index
<code>df.combine_first()</code>	Combine with priority

Time Series

Function / Method	Description
<code>pd.to_datetime()</code>	Parse date strings
<code>dt.year / dt.month</code>	Extract date parts
<code>df.resample('M')</code>	Monthly aggregation
<code>df.rolling(n)</code>	Rolling window
<code>df.shift(n)</code>	Lag / lead values